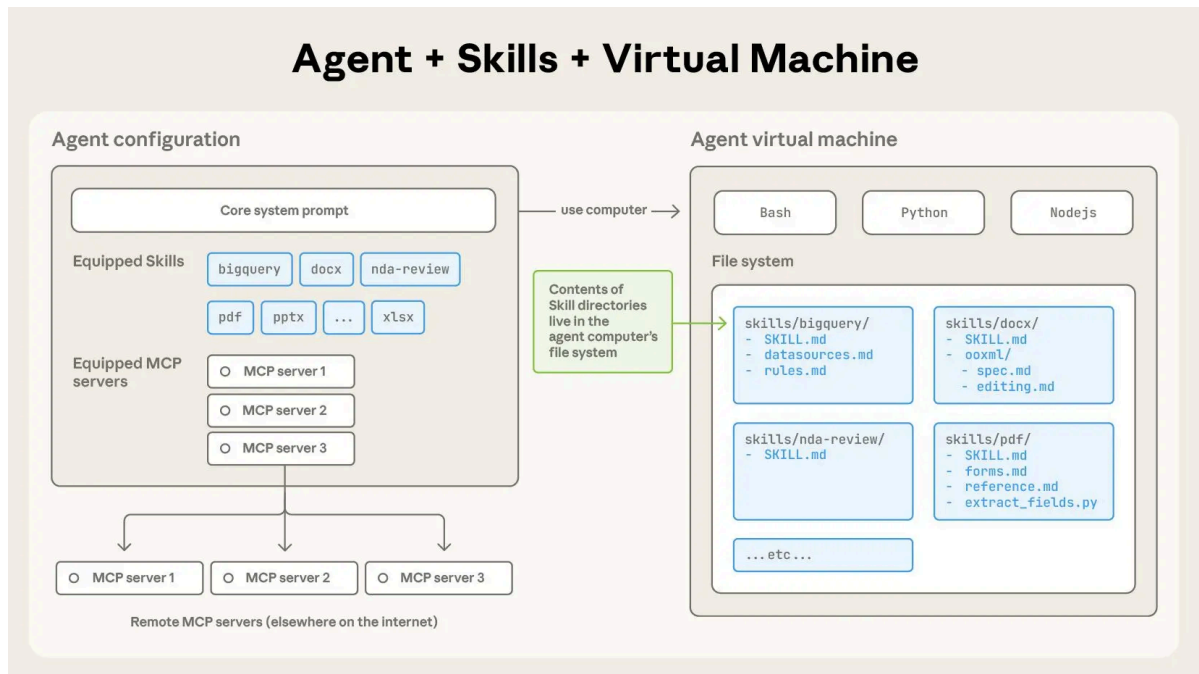


Agent Skills



Skill 是一个目录，其中包含一个名为 SKILL.md 的文件。该文件包含组织有序的指令文件夹、脚本文件夹和资源文件夹，为智能体提供额外功能。

一、MCP 之后，我们还需要什么？

MCP (Model Context Protocol) 由 Anthropic 团队提出，其核心设计理念是**标准化智能体与外部工具/资源的通信方式**。想象一下，你的智能体需要访问文件系统、数据库、GitHub、Slack 等各种服务。传统做法是为每个服务编写专门的适配器，这不仅工作量大，而且难以维护。MCP 通过定义统一的协议规范，让所有服务都能以相同的方式被访问。

MCP 的设计哲学是"上下文共享"。它不仅仅是一个远程过程调用协议，更重要的是它允许智能体和工具之间共享丰富的上下文信息。让我们回顾一个典型的 MCP 使用场景：

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_mcp_adapters.client import MultiServerMCPClient

client = MultiServerMCPClient(
    {
        "db-emp": {
            "transport": "stdio",
            "command": "python",
            "args": ["database_mcp_server.py"],
        }
    }
)

# 一次性加载 MCP Server 所有的工具（动态发现）
tools = await client.get_tools()
```

```
model = ChatOpenAI(model="gpt-5")

# ReAct Agent
agent = create_agent(
    model,
    tools
)

result = agent.invoke("查询员工表中薪资最高的前10名员工")
```

这段代码工作得很好，智能体成功连接到了数据库。但当你尝试处理更复杂的任务时，会发现一些微妙的问题：

```
# 一个更复杂的需求
response = agent.invoke(
    """
    分析公司内部谁的话语权最高？ 需要综合考虑：
    1. 管理层级和下属数量
    2. 薪资水平和涨薪幅度
    3. 任职时长和稳定性
    4. 跨部门影响力
    """
)
```

这个任务需要执行多次数据库查询，每次查询的结果会影响下一次查询的策略。更关键的是，它需要智能体具备**领域知识**：知道如何衡量"话语权"，知道应该从哪些维度分析数据，知道如何组合多个查询结果得出结论。

此时，你会遇到两个根本性的问题：

第一个问题是上下文爆炸。为了让智能体能够灵活查询数据库，MCP 服务器通常会暴露数十甚至上百个工具（不同的表、不同的查询方法）。这些工具的完整 JSON Schema 在连接建立时就会被加载到系统提示词中，可能占用数万个 token。据社区开发者反馈，仅加载一个 Playwright MCP 服务器就会占用 200k 上下文窗口的 8%，这在多轮对话中会迅速累积，导致成本飙升和推理能力下降。

第二个问题是能力鸿沟。MCP 解决了"能够连接"的问题，但没有解决"知道如何使用"的问题。拥有数据库连接能力，不等于智能体知道如何编写高效且安全的 SQL；能够访问文件系统，不意味着它理解特定项目的代码结构和开发规范。这就像给一个新手程序员开通了所有系统的访问权限，但没有提供操作手册和最佳实践。

这正是 **Agent Skills** 要解决的核心问题。2025年初，Anthropic 在推出 MCP 之后，进一步提出了 Agent Skills 的概念，引发了业界的广泛关注。有开发者评论说："Skill 和 MCP 是两种东西，Skill 是领域知识，告诉模型该如何做，本质上是高级 Prompt；而 MCP 对接外部工具和数据。"也有人认为："从 Function Call 到 Tool Call 到 MCP 到 Skill，核心大差不差，就是工程实践和表现形式的优化演进。"

那么，Agent Skills 到底是什么？它与 MCP 有何本质区别？两者是竞争关系还是互补关系？。

二、什么是 Agent Skills?

2.1 Agent Skills 基本概念

Agent Skills 是一种轻量级的开放格式，用于通过专业知识和工作流扩展 AI 智能体的能力。与 MCP 服务器需要运行代码服务不同，一个"技能" (Skill) 在形式上就是一个结构化的文件夹。其核心是一个名为 `SKILL.md` 的文件，包含了元数据（如名称和描述）以及告知智能体如何执行特定任务的详细指令。

Skill 为智能体打包领域专业知识和程序知识。

标准的目录结构如下：

```
my-skill/
├── SKILL.md      # 核心文件：元数据+指令
├── scripts/      # 可选：该技能专属的可执行代码
├── references/   # 可选：技术文档，领域知识参考
└── assets/      # 可选：模板、静态资源
```

为了支持复杂的任务，除了核心的 `SKILL.md`，Skill 文件夹还可以包含以下可选目录：

- `scripts/`：包含该技能专属的自动化脚本（如 Python、Bash 或 Node.js）。当某些任务通过纯 Prompt 难以稳定实现，或者需要进行复杂的数值计算、文件处理时，Agent 可以直接运行这些脚本。
- `references/`：存放该领域的专业文档、API 手册、技术标准或常见问题解答（FAQ）。Agent 会在需要时按需读取这些参考资料，这既能保证专业性，又避免了将所有知识硬编码在提示词中。
- `assets/`：存放各种静态资源，例如配置模板、数据文件、用于对比的示例图像或预定义的 JSON Schema。这些资源为 Agent 提供了执行任务所需的"原材料"。

如果说 MCP 是一个个独立的工具，那么 Skill 就是一个工具箱——里面不仅装有工具，还附带了详细的使用说明书（`SKILL.md`）。

[Building Agents with Skills: Equipping Agents for Specialized Work | Claude](#)

Agent Skills 开放标准：<https://agentskills.io/home>

2.2 为什么需要 Agent Skills?

随着智能助手的能力日益增强，但往往缺乏可靠完成实际工作所需的上下文信息。技能模块通过赋予助手按需加载程序化知识及企业、团队和用户专属上下文的能力，有效解决了这一问题。具备技能集的智能助手可根据当前任务动态扩展功能。

- 技能开发者：一次构建能力，即可部署至多个智能体产品。
- 兼容智能体：技能支持让终端用户开箱即可赋予智能体新能力。
- 团队与企业：将组织知识封装为可移植、版本控制的知识包。

2.3 核心设计理念

Agent Skills 是一种标准化的程序性知识封装格式。如果说 MCP 为智能体提供了"手"来操作工具，那么 Skills 就提供了"操作手册"或"SOP（标准作业程序）"，教导智能体如何正确使用这些工具。

这种设计理念源于一个简单但深刻的洞察：**连接性（Connectivity）与能力（Capability）应该分离。**MCP 专注于前者，Skills 专注于后者。这种职责分离带来了清晰的架构优势：

- **MCP 的职责**：提供标准化的访问接口，让智能体能够"够得着"外部世界的数据和工具

- **Skills 的职责**：提供领域专业知识，告诉智能体在特定场景下"如何组合使用这些工具"

用一个类比来理解：MCP 像是 USB 接口或驱动程序，它定义了设备如何连接；而 Skills 像是软件应用程序，它定义了如何使用这些连接的设备来完成具体任务。你可以拥有一个功能完善的打印机驱动（MCP），但如果没有告诉你如何在 Word 里设置页边距和双面打印（Skill），你仍然无法高效地完成打印任务。

2.4 Agent Skills 的优势

- **自文档化 (Self-documenting)**：开发者和用户都可以直接阅读 `SKILL.md`，轻松理解其逻辑。
- **可移植性 (Portable)**：基于文件系统，可以在不同的智能体平台（如 Claude Code, Cursor 等）之间无缝迁移。
- **低门槛**：只要你会写 Markdown 和简单的脚本，你就能为 Agent 创造一项新技能。
- **高效性**：通过渐进式披露机制，在保持强大功能的同时最小化上下文占用。
- **模块化**：技能可以独立开发、测试和分发，便于团队协作和知识积累。

2.5 渐进式披露：破解上下文困境

Agent Skills 最核心的创新是**渐进式披露 (Progressive Disclosure) 机制**。这种机制将技能信息分为三个层次，智能体按需逐步加载，既确保必要时不遗漏细节，又避免一次性将过多内容塞入上下文窗口。

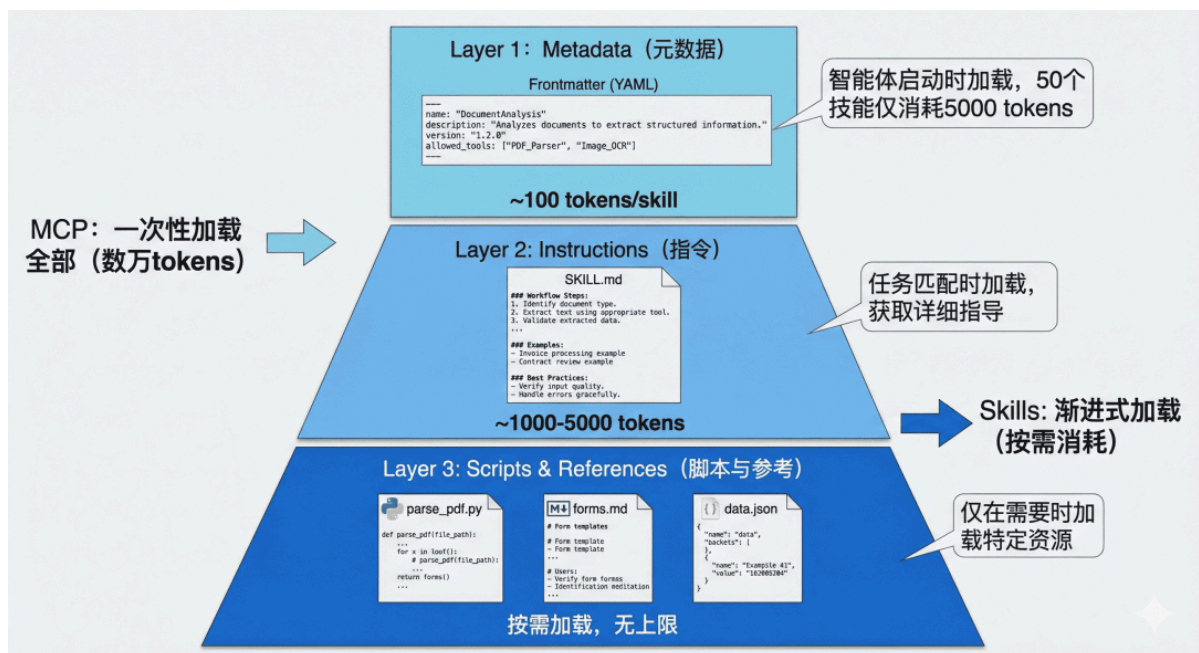


图 1 Agent Skills 渐进式披露三层架构

第一层：元数据 (Metadata)

在 Skills 的设计中，每个技能都存放在一个独立的文件夹中，核心是一个名为 `SKILL.md` 的 Markdown 文件。这个文件必须以 YAML 格式的 Frontmatter 开头，定义技能的基本信息。

```
---
name: Anthropic Brand Style Guidelines
description: Anthropic's official brand colors and typography...
---
```

当智能体启动时，它会扫描所有已安装的技能文件夹，**仅读取每个 SKILL.md 的 Frontmatter 部分**，将这些元数据加载到系统提示词中。根据实测数据，每个技能的元数据仅消耗约**100 个 token**。即使你安装了 50 个技能，初始的上下文消耗也只有约 5,000 个 token。

这与 MCP 的工作方式形成了鲜明对比。在典型的 MCP 实现中，当客户端连接到一个服务器时，通常会通过 `tools/list` 请求获取所有可用工具的完整 JSON Schema，可能立即消耗数万个 token。

第二层：技能主体 (Instructions)

当智能体通过分析用户请求，判断某个技能与当前任务高度相关时，它会进入第二层加载。此时，智能体会读取该技能的完整 SKILL.md 文件内容，将详细的指令、注意事项、示例等加载到上下文中。

此时，智能体获得了完成任务所需的全部上下文：数据库结构、查询模式、注意事项等。这部分内容的 token 消耗取决于指令的复杂度，通常在 1,000 到 5,000 个 token 之间。

第三层：附加资源 (Scripts & References)

对于更复杂的技能，SKILL.md 可以引用同一文件夹下的其他文件：脚本、配置文件、参考文档等。智能体**仅在需要时才加载这些资源**。

例如，一个 PDF 处理技能的文件结构可能是：

```
skills/pdf-processing/
├─ SKILL.md # 主技能文件
├─ parse_pdf.py # PDF 解析脚本
├─ forms.md # 表单填写指南（仅在填表任务时加载）
├─ templates/ # PDF 模板文件
│   └─ invoice.pdf
│   └─ report.pdf
```

在 SKILL.md 中，可以这样引用附加资源：

- 当需要执行 PDF 解析时，智能体会运行 `parse_pdf.py` 脚本
- 当遇到表单填写任务时，才会加载 `forms.md` 了解详细步骤
- 模板文件只在需要生成特定格式文档时访问

这种设计有两个关键优势：

1. **无限的知识容量**：通过脚本和外部文件，技能可以“携带”远超上下文限制的知识。例如，一个数据分析技能可以附带一个 1GB 的数据文件和一个查询脚本，智能体通过执行脚本来访问数据，而无需将整个数据集加载到上下文中。
2. **确定性执行**：复杂的计算、数据转换、格式解析等任务交给代码执行，避免了 LLM 生成过程中的不确定性和幻觉问题。
3. 渐进式披露的效果：从 16k 到 500 Token

社区开发者分享的实践案例充分证明了渐进式披露的威力。在一个真实场景中：

- **传统 MCP 方式**：直接连接一个包含大量工具定义的 MCP 服务器，初始加载消耗 **16,000 个 token**
- **Skills 包装后**：创建一个简单的 Skill 作为“网关”，仅在 Frontmatter 中描述功能，初始消耗仅 **500 个 token**

当智能体确定需要使用该技能时，才会加载详细指令并按需调用底层的 MCP 工具。这种架构不仅大幅降低了初始成本，还使得对话过程中的上下文管理更加精准和高效。

2.6 Skills 完整架构

综合来看，新兴的智能体架构呈现出以下组合形态：

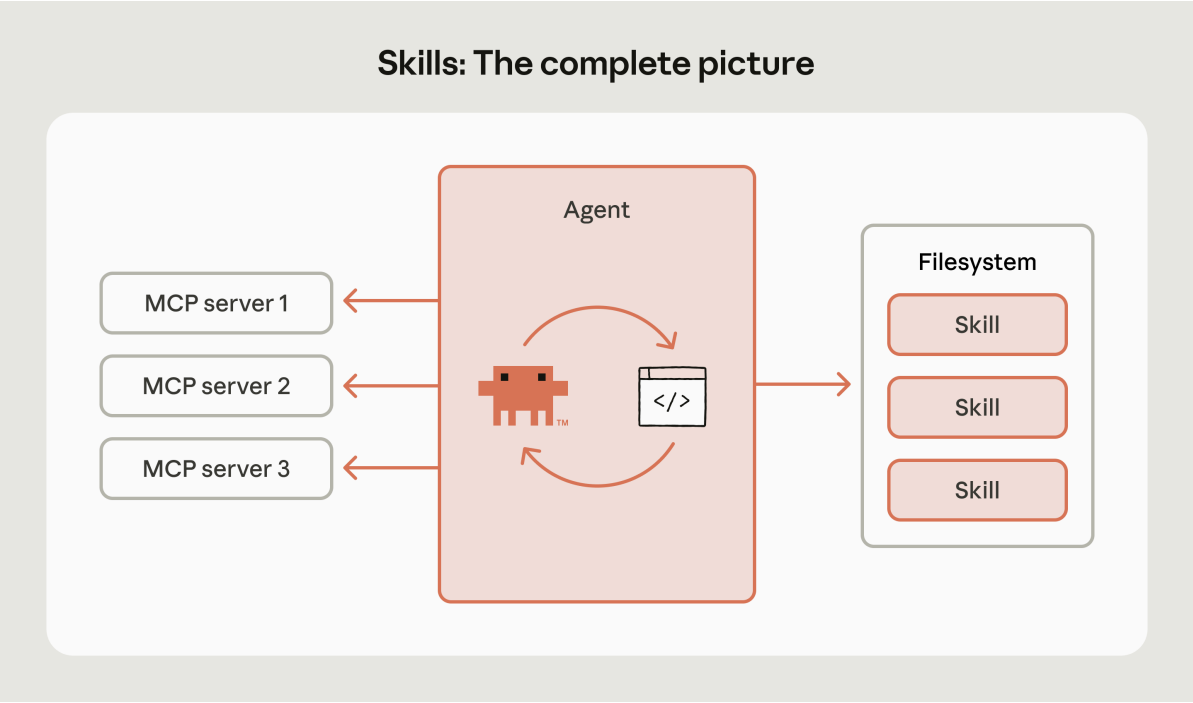
1. 智能体循环（**Agent loop**）：决定下一步行动的核心推理系统

智能体循环是使人工智能智能体持续朝着目标努力的循环过程。首先，智能体从其工具、传感器或记忆中收集最新数据。随后更新内部状态并制定行动方案，通常通过执行规划或推理步骤实现。接着执行选定操作，例如调用API、写入文件或发送消息。行动后检查结果并存储新信息。循环将基于最新数据重新启动，使智能体能够适应变化并持续进化。这种观察-决策-行动的快速迭代赋予了智能体强大能力。

2. 智能体运行时（**Agent runtime**）：执行环境（代码、文件系统）

3. MCP服务器（**MCP servers**）：连接外部工具与数据源的接口

4. 技能库（**Skills library**）：领域专业知识与程序化知识的集合



每层都具有明确的功能：

- 循环负责推理
- 运行时负责执行
- MCP负责连接
- 技能负责引导

这种分离使系统易于理解，并允许各部分独立演进。

试想当你向这个架构添加一项技能时会发生什么。前端设计技能能瞬间提升Claude的前端能力，在构建网页界面时自动激活，提供排版、色彩理论和动画等专业指导。渐进式披露机制确保其仅在相关场景下加载。新增功能的集成过程也极为简便。

三、Agent Skills vs MCP：本质区别与协作关系

现在，我们可以系统地比较这两种技术的本质区别了。

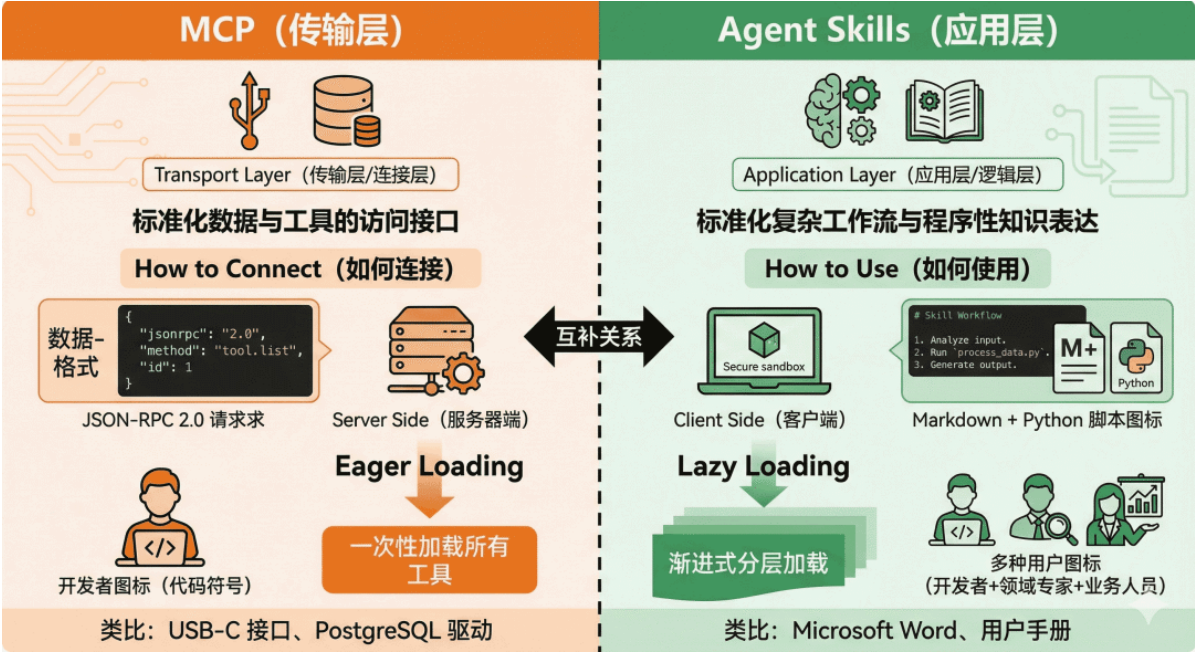


图 2 MCP 与 Agent Skills 设计哲学对比

3.1 从工程视角理解差异

让我们通过一个具体的例子来理解这种差异。假设你要构建一个智能体来帮助团队进行代码审查：

MCP 的职责：

```
# MCP 提供对 GitHub 的标准化访问
github_mcp = MCPTool(server_command=["npx", "-y", "@modelcontextprotocol/server-github"])

# MCP 暴露的工具（简化示例）：
# - list_pull_requests(repo, state)
# - get_pull_request_details(pr_number)
# - list_pr_comments(pr_number)
# - create_pr_comment(pr_number, body)
# - get_file_content(repo, path, ref)
# - list_pr_files(pr_number)
```

MCP 让智能体"能够"访问 GitHub，能够调用这些 API。但它不知道"应该"做什么。

Skills 的职责：

```
---
name: code-review-workflow
description: 执行标准的代码审查流程，包括检查代码风格、安全问题、测试覆盖率等。
---

# 代码审查工作流

## 审查清单 当执行代码审查时，按以下步骤进行：

1. **获取 PR 信息**：调用 `get_pull_request_details` 了解变更背景
2. **分析变更文件**：调用 `list_pr_files` 获取文件列表
3. **逐文件审查**：
  - 对于 `.py` 文件：检查是否符合 PEP 8，是否有明显的性能问题
  - 对于 `.js/.ts` 文件：检查是否有未处理的 Promise，是否使用了废弃的 API
```

- 对于测试文件：验证是否覆盖了新增的代码路径
- 4. ****安全检查****：
 - 是否硬编码了敏感信息（密钥、密码）
 - 是否有 SQL 注入或 XSS 风险
- 5. ****提供反馈****：
 - 严重问题：使用 ``create_pr_comment`` 直接评论
 - 建议改进：在总结中提出

公司特定规范

- 所有数据库查询必须使用参数化查询
- API 端点必须有权限验证装饰器
- 新功能必须附带单元测试（覆盖率 > 80%）

示例评论模板

****严重问题****: ⚠️ 安全风险：第 45 行直接拼接 SQL 字符串，存在注入风险。建议改用参数化查询：
``cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))``

Skills 告诉智能体"应该"做什么、如何组织审查流程、需要关注哪些公司特定的规范。它是领域知识和最佳实践的容器。

3.2 上下文管理策略的本质差异

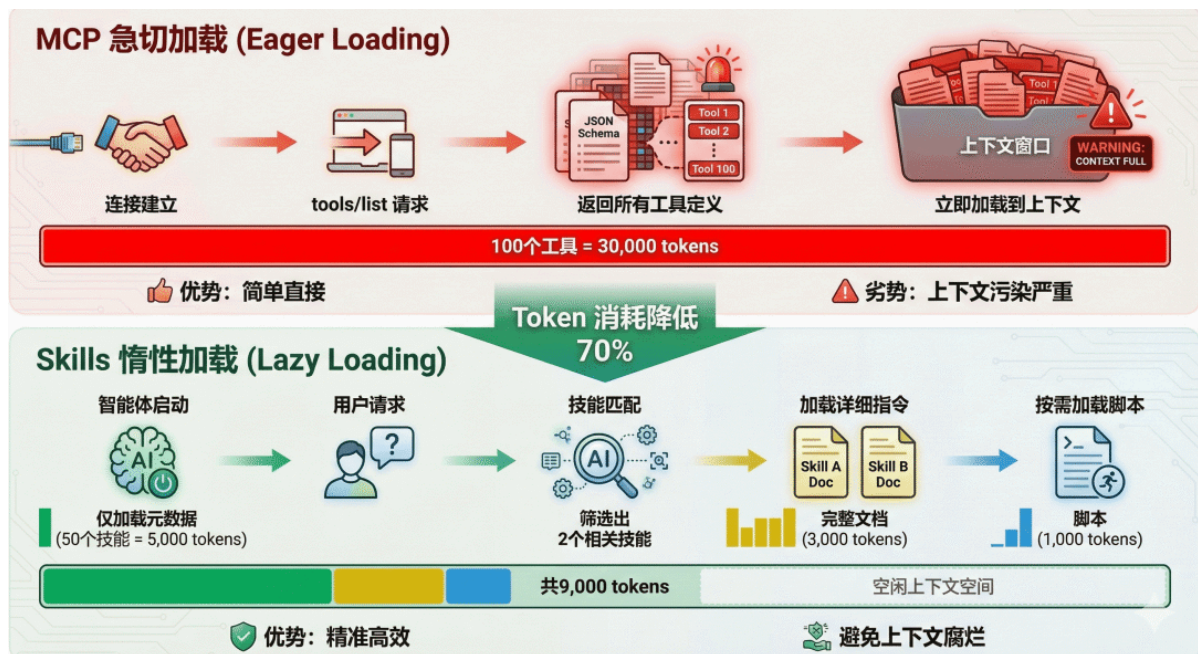


图 3 MCP 急切加载 vs Skills 惰性加载对比

3.3 互补而非竞争：Skills + MCP 的混合架构

理解了两者的差异后，我们会发现：Skills 和 MCP 不是竞争关系，而是互补关系。最佳实践是将两者结合，形成分层架构：

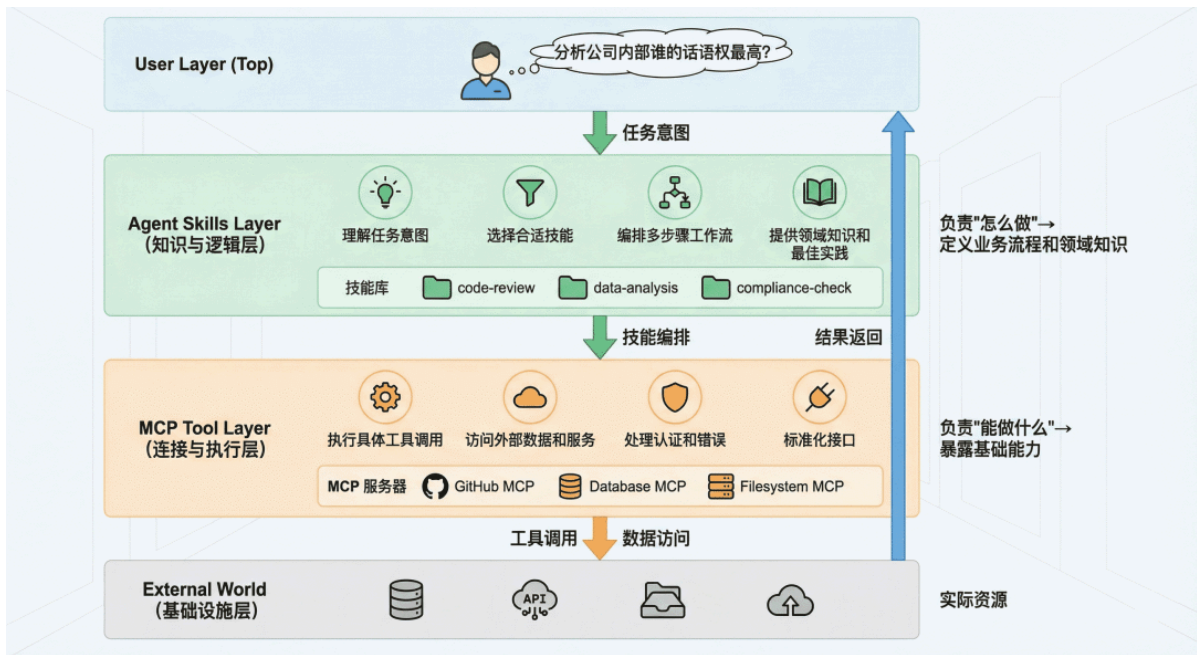


图 4 Skills + MCP 混合架构设计

典型工作流：

1. 用户问："分析公司内部谁的话语权最高"
2. **Skills 层**识别这是一个数据分析任务，加载 `mysql-employees-analysis` 技能
3. **Skills 层**根据技能指令，将任务分解为子步骤：查询管理关系、薪资对比、任职时长等
4. **MCP 层**执行具体的 SQL 查询，返回结果
5. **Skills 层**根据技能中的领域知识，解读数据并生成综合分析
6. 返回结构化的答案给用户

这种架构的优势是：

- **关注点分离**：MCP 专注于"能力"，Skills 专注于"智慧"
- **成本优化**：渐进式加载大幅降低 token 消耗
- **可维护性**：业务逻辑（Skills）与基础设施（MCP）解耦
- **复用性**：同一个 MCP 服务器可以被多个 Skills 使用

四、技术实现：如何创建和使用 Skills

4.1. SKILL.md 规范详解

[Specification - Agent Skills](#)

让我们深入了解 `SKILL.md` 文件的标准结构：

```
---
# === 必需字段 ===
name: skill-name # 技能的唯一标识符，使用 kebab-case 命名
description: 简洁但精确的描述，说明： 1.这个技能做什么 2.什么时候应该使用它 3.它的核心价值是什么 # 注意: description 是智能体选择技能的唯一依据，必须写清楚！
# === 可选字段 ===
version: 1.0.0 # 版本号
```

```
allowed_tools: [tool1, tool2] # 此技能可以调用的工具列表（白名单）
required_context: [context_item1] # 此技能需要的上下文信息
license: MIT # 许可协议
author: Your Name # 作者信息
tags: [database, analysis, sql] # 便于分类和搜索的标签
---
```

```
# 技能标题
## 概述（对技能的详细介绍，包括使用场景、技术背景等）
## 前置条件（使用此技能需要的环境配置、依赖项等）
## 工作流程（详细的步骤说明，告诉智能体如何执行任务）
## 最佳实践（经验总结、注意事项、常见陷阱等）
## 示例（具体的使用案例，帮助智能体理解）
## 故障排查（常见问题和解决方案）
```

4.2 编写高质量 Skills 的原则

根据 Anthropic 官方文档和社区最佳实践，编写有效的 Skills 需要遵循以下原则：

1. 精准的 Description

`description` 是智能体决策的关键。它应该：

- **精确定义适用范围**：避免模糊的描述如"帮助处理数据"
- **包含触发关键词**：让智能体能够匹配用户意图
- **说明独特价值**：与其他技能区分开来

✗ 不好的 description：

```
description: 处理数据库查询
```

✓ 好的 description：

```
description: 将中文业务问题转换为 SQL 查询并分析 MySQL employees 示例数据库。适用于员工信息查询、薪资统计、部门分析、职位变动历史等场景。当用户询问关于员工、薪资、部门的数据时使用此技能。
```

2. 模块化与单一职责

一个 Skill 应该专注于一个明确的领域或任务类型。如果一个 Skill 试图做太多事情，会导致：

- Description 过于宽泛，匹配精度下降
- 指令内容过长，浪费上下文
- 难以维护和更新

建议：与其创建一个"通用数据分析"技能，不如创建多个专门的技能：

- `mysql-employees-analysis`：专门分析 employees 数据库
- `sales-data-analysis`：专门分析销售数据
- `user-behavior-analysis`：专门分析用户行为数据

4.3 确定性优先原则

对于复杂的、需要精确执行的任务，优先使用脚本而不是依赖 LLM 生成。例如，在数据导出场景中，与其让 LLM 生成 Excel 二进制内容（容易出错），不如编写一个专门的脚本来处理这个任务，SKILL.md 中只需要指导智能体何时调用这个脚本即可。

4.4 渐进式披露策略

合理利用三层结构，将信息按重要性和使用频率分层：

- **SKILL.md 主体**：放置核心 workflow、常用模式
- **附加文档**（如 advanced.md）：放置高级用法、边缘情况
- **数据文件**：放置大型参考数据，通过脚本按需查询

五、Claude Code Skills 实战

5.1 安装与配置

安装并配置好 Claude Code

1. 安装 Claude Code: <https://code.claude.com/docs/zh-CN/overview>

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

2. 配置模型 Model

手动配置适用于所有兼容 Anthropic API 的模型。配置方式如下：

Windows:

```
setx ANTHROPIC_BASE_URL "模型API地址"
setx ANTHROPIC_AUTH_TOKEN "你的API密钥"
setx ANTHROPIC_MODEL "模型名称"
```

macOS/Linux:

```
export ANTHROPIC_BASE_URL=模型API地址
export ANTHROPIC_AUTH_TOKEN=你的API密钥
export ANTHROPIC_MODEL=模型名称

# 永久配置(添加到 ~/.bashrc 或 ~/.zshrc)
echo 'export ANTHROPIC_BASE_URL=模型API地址' >> ~/.bashrc
echo 'export ANTHROPIC_AUTH_TOKEN=你的API密钥' >> ~/.bashrc
echo 'export ANTHROPIC_MODEL=模型名称' >> ~/.bashrc
source ~/.bashrc
```

常用国内模型配置示例:

模型	API地址	模型名称	获取API Key
智谱 GLM-4.7	<code>https://open.bigmodel.cn/api/anthropic</code>	<code>glm-4.7</code>	https://open.bigmodel.cn/
Kimi K2	<code>https://api.moonshot.cn/anthropic</code>	<code>kimi-k2-turbo-preview</code>	https://platform.moonshot.cn/console/account
通义千问	<code>https://dashscope.aliyuncs.com/apps/anthropic</code>	<code>qwen-coder-plus</code>	https://bailian.console.aliyun.com/
DeepSeek	<code>https://api.deepseek.com/anthropic</code>	<code>deepseek-chat</code>	https://platform.deepseek.com/

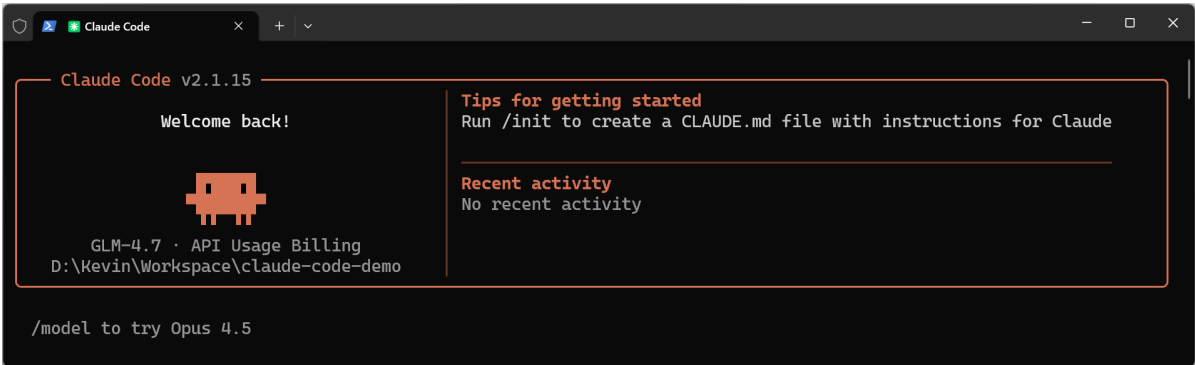
配置示例(以智谱GLM为例):

```
export ANTHROPIC_BASE_URL=https://open.bigmodel.cn/api/anthropic
export ANTHROPIC_AUTH_TOKEN=your_glm_api_key
export ANTHROPIC_MODEL=GLM-4.7
```

注意: 配置环境变量后需要重启终端或运行 `source ~/.bashrc` 使配置生效。

3. 登录 Claude 账户

4. 启动 Claude Code



5.2 Anthropic Skills

<https://github.com/anthropics/skills>

您可以在Claude Code中运行以下命令，将此存储库注册为Claude Code插件市场：

```
/plugin marketplace add anthropics/skills
```

然后，要安装特定技能组：

1. Select `Browse` and `install plugins`
2. Select `anthropic-agent-skills`
3. Select `document-skills` or `example-skills`
4. Select `Install now`

或者，直接通过以下方式安装任一插件：

```
/plugin install document-skills@anthropic-agent-skills
/plugin install example-skills@anthropic-agent-skills
```

安装插件后，只需提及该技能即可使用。例如，若从应用商店安装了文档技能插件，可向Claude Code发出如下指令：“使用PDF skill 从 test.pdf 中提取表格数据”。

示例：使用 pdf skill 提取 test.pdf 中的表格数据

```
使用 pdf skill 提取 test.pdf 中的表格数据
• I'll use the PDF skill to extract table data from your test.pdf file.
• Skill(pdf)
  | Successfully loaded skill
• I'll help you extract table data from test.pdf using the PDF skill. Let me first check if the file exists and then extract the tables.
• Bash(ls -la test.pdf)
  | -rw-r--r-- 1 Kevin 197121 594846 1月 29 16:51 test.pdf
• Now I'll extract the tables from test.pdf using pdfplumber, which is specifically designed for table extraction:
• Bash(python -c "
  | import pdfplumber...)
  | Error: Exit code 1
  | Traceback (most recent call last):
  |   File "<string>", line 3, in <module>
  |     import pandas as pd
  | ModuleNotFoundError: No module named 'pandas'
• Let me try a simpler approach without pandas:
• Bash(python -c "
  | import pdfplumber...)
  | Extracting tables from test.pdf...
  | Found table 1 on page 3 with 7 rows
  | ... +7 lines (ctrl+o to expand)
• I found one table with 7 rows on page 3 of test.pdf. Let me also check the content of the extracted CSV file:
• Bash(type table_1.csv)
  | Error: Exit code 1
  | /usr/bin/bash: line 1: type: table_1.csv: not found
• Bash(cat table_1.csv)
  | 阶段,目标,工程重点
  | 模型消费,通过 API 调用模型服务,掌握 SDK / Token 管理
  | 上下文工程,提高提示词与输出稳定性,Prompt 模板化 / PromptOps
  | ... +4 lines (ctrl+o to expand)
• Table data has been successfully extracted from test.pdf. Here's the summary:

Extraction Results

Found 1 table on page 3 with 7 rows

The table has been saved to table_1.csv and contains:


| 阶段    | 目标                | 工程重点                             |
|-------|-------------------|----------------------------------|
| 模型消费  | 通过 API 调用模型服务     | 掌握 SDK / Token 管理                |
| 上下文工程 | 提高提示词与输出稳定性       | Prompt 模板化 / PromptOps           |
| 知识增强  | 集成检索与向量数据库        | RAG / Embedding / Index          |
| 工具协作  | 引入多 Agent 与外部系统执行 | Agentic Framework / MCP / Skills |
| 系统工程化 | 部署可观测的 AI 系统      | GPU 编排 / AIOps / Gateway         |
| 领域优化  | 微调与持续改进           | LoRA / Evaluation / Monitoring   |


The extracted data has been saved to table_1.csv in your working directory.
* Churned for 1m 49s
```

提取的结果

5.3 自定义 Skills

详见 `weather-skill.html` 项目文件

六、LangChain Agent Skills 实战

本章节展示如何运用渐进式披露（一种情境管理技术），代理通过按需加载信息而非预先加载来实现技能（基于提示的专项指令）。代理通过工具调用加载技能，而非动态更改系统提示，仅针对每项任务发现并加载所需技能。

说明

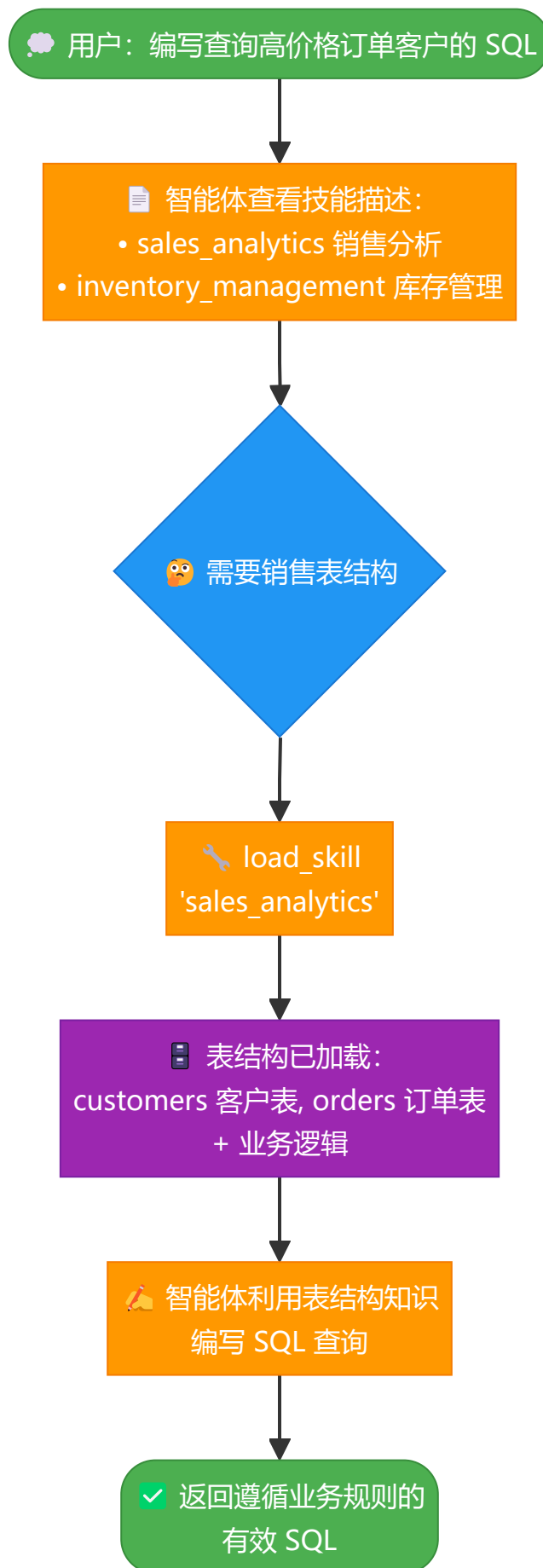
渐进式信息披露由Anthropic公司推广，作为构建可扩展智能体技能系统的技术方案。该方法采用三级架构（元数据→核心内容→详细资源），智能体仅按需加载信息。有关此技术的更多内容，请参阅[Equipping agents for the real world with Agent Skills](#)。

用例：构建一个智能助手，用于协助大型企业跨不同业务领域编写SQL查询。企业可能为每个业务领域配置独立数据存储，也可能采用包含数千张表的单体数据库。无论哪种情况，预先加载所有模式都会导致上下文窗口不堪重负。渐进式加载机制通过按需加载相关模式解决了这一问题。该架构还支持不同产品负责人和利益相关者独立贡献并维护各自业务领域的技能库。

开始构建：一个具备两项技能（销售分析与库存管理）的SQL查询助手。该智能体在系统提示中仅显示轻量级技能描述，仅当用户查询涉及相关业务时，才通过工具调用加载完整的数据库架构和业务逻辑。

6.1 工作原理

当用户请求SQL查询时，流程如下：



为何采用渐进式披露：

- 减少上下文消耗：仅加载任务所需的2-3项技能，而非全部可用技能

- 赋能团队自主性：不同团队可独立开发专业技能
- 高效扩展：添加数十或数百项技能而不致上下文过载
- 简化对话历史：单一智能体对应单一对话线程

Skill 是什么：由Claude Code推广的技能，本质上是以提示为基础的：它们是为特定业务任务设计的、自包含的专用指令单元。在Claude Code中，技能以文件系统目录形式呈现，通过文件操作进行发现。技能通过提示引导行为，既可提供工具使用说明，也可包含供编码代理执行的示例代码。

说明

具有渐进式披露功能的技能可视为一种检索增强生成（RAG）形式，其中每个技能即为检索单元，尽管其检索机制未必依赖嵌入向量或关键词搜索，而是通过内容浏览工具实现（如文件操作）。

设计思想：



6.2 环境配置

```
conda create -n langchain-agent-skills python=3.11
conda activate langchain-agent-skills

pip install -U "langchain[openai]"
```

6.3 完整实现

Skill 结构



Tables (数据库表结构)

customers	orders	order_items
├ customer_id(PK)	├ order_id(PK)	├ item_id(PK)
├ name	├ customer_id(FK)	├ order_id(FK)
├ email	├ order_date	├ product_id
├ signup_date	├ status	├ quantity
├ status	├ total_amount	├ unit_price
└ customer_tier	└ sales_region	└ discount

Business Logic (业务逻辑)

- **Active customers** 的定义
- **Revenue** 如何计算
- **High-value orders** 的阈值

Example Query (示例查询)

- 提供参考模板帮助 LLM 理解风格

```
---
name: "sales_analytics"
"description": "Database schema and business logic for sales data analysis
including customers, orders, and revenue."
---
```

Sales Analytics Schema

Tables

customers

- customer_id (PRIMARY KEY)
- name
- email
- signup_date
- status (active/inactive)
- customer_tier (bronze/silver/gold/platinum)

orders

- order_id (PRIMARY KEY)
- customer_id (FOREIGN KEY -> customers)
- order_date
- status (pending/completed/cancelled/refunded)
- total_amount
- sales_region (north/south/east/west)

order_items

- item_id (PRIMARY KEY)
- order_id (FOREIGN KEY -> orders)
- product_id
- quantity
- unit_price
- discount_percent

Business Logic

****Active customers****: status = 'active' AND signup_date <= CURRENT_DATE - INTERVAL '90 days'

****Revenue calculation****: Only count orders with status = 'completed'. Use total_amount from orders table, which already accounts for discounts.

****Customer lifetime value (CLV)****: Sum of all completed order amounts for a customer.

****High-value orders****: Orders with total_amount > 1000

Example Query

```
-- Get top 10 customers by revenue in the last quarter
SELECT
    c.customer_id,
    c.name,
    c.customer_tier,
    SUM(o.total_amount) as total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
    AND o.order_date >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY c.customer_id, c.name, c.customer_tier
ORDER BY total_revenue DESC
LIMIT 10;
```

完整代码实现

```
import uuid
from typing import TypedDict, NotRequired
from langchain.tools import tool
from langchain.agents import create_agent
from langchain.agents.middleware import ModelRequest, ModelResponse, AgentMiddleware
from langchain.messages import SystemMessage
from langgraph.checkpoint.memory import InMemorySaver
from typing import Callable

# Define skill structure
class Skill(TypedDict):
    """A skill that can be progressively disclosed to the agent."""
    name: str
    description: str
    content: str

# Define skills with schemas and business logic
SKILLS: list[Skill] = [
    {
        "name": "sales_analytics",
        "description": "Database schema and business logic for sales data analysis including customers, orders, and revenue.",
    },
]
```

```

"content": """"# Sales Analytics Schema

## Tables

### customers
- customer_id (PRIMARY KEY)
- name
- email
- signup_date
- status (active/inactive)
- customer_tier (bronze/silver/gold/platinum)

### orders
- order_id (PRIMARY KEY)
- customer_id (FOREIGN KEY -> customers)
- order_date
- status (pending/completed/cancelled/refunded)
- total_amount
- sales_region (north/south/east/west)

### order_items
- item_id (PRIMARY KEY)
- order_id (FOREIGN KEY -> orders)
- product_id
- quantity
- unit_price
- discount_percent

## Business Logic

**Active customers**: status = 'active' AND signup_date <= CURRENT_DATE -
INTERVAL '90 days'

**Revenue calculation**: Only count orders with status = 'completed'. Use
total_amount from orders table, which already accounts for discounts.

**Customer lifetime value (CLV)**: Sum of all completed order amounts for a
customer.

**High-value orders**: Orders with total_amount > 1000

## Example Query

-- Get top 10 customers by revenue in the last quarter
SELECT
    c.customer_id,
    c.name,
    c.customer_tier,
    SUM(o.total_amount) as total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'completed'
    AND o.order_date >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY c.customer_id, c.name, c.customer_tier
ORDER BY total_revenue DESC
LIMIT 10;
"""

```

```

    """
    },
    {
        "name": "inventory_management",
        "description": "Database schema and business logic for inventory tracking
including products, warehouses, and stock levels.",
        "content": """# Inventory Management Schema

## Tables

### products
- product_id (PRIMARY KEY)
- product_name
- sku
- category
- unit_cost
- reorder_point (minimum stock level before reordering)
- discontinued (boolean)

### warehouses
- warehouse_id (PRIMARY KEY)
- warehouse_name
- location
- capacity

### inventory
- inventory_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- quantity_on_hand
- last_updated

### stock_movements
- movement_id (PRIMARY KEY)
- product_id (FOREIGN KEY -> products)
- warehouse_id (FOREIGN KEY -> warehouses)
- movement_type (inbound/outbound/transfer/adjustment)
- quantity (positive for inbound, negative for outbound)
- movement_date
- reference_number

## Business Logic

**Available stock**: quantity_on_hand from inventory table where quantity_on_hand
> 0

**Products needing reorder**: Products where total quantity_on_hand across all
warehouses is less than or equal to the product's reorder_point

**Active products only**: Exclude products where discontinued = true unless
specifically analyzing discontinued items

**Stock valuation**: quantity_on_hand * unit_cost for each product

## Example Query

```

```

-- Find products below reorder point across all warehouses
SELECT
    p.product_id,
    p.product_name,
    p.reorder_point,
    SUM(i.quantity_on_hand) as total_stock,
    p.unit_cost,
    (p.reorder_point - SUM(i.quantity_on_hand)) as units_to_reorder
FROM products p
JOIN inventory i ON p.product_id = i.product_id
WHERE p.discontinued = false
GROUP BY p.product_id, p.product_name, p.reorder_point, p.unit_cost
HAVING SUM(i.quantity_on_hand) <= p.reorder_point
ORDER BY units_to_reorder DESC;
"""
    },
]

# Create skill loading tool
@tool
def load_skill(skill_name: str) -> str:
    """Load the full content of a skill into the agent's context.

    Use this when you need detailed information about how to handle a specific
    type of request. This will provide you with comprehensive instructions,
    policies, and guidelines for the skill area.

    Args:
        skill_name: The name of the skill to load (e.g., "sales_analytics",
"inventory_management")
    """

    # Find and return the requested skill
    for skill in SKILLS:
        if skill["name"] == skill_name:
            return f"Loaded skill: {skill_name}\n\n{skill['content']}"

    # Skill not found
    available = ", ".join(s["name"] for s in SKILLS)
    return f"Skill '{skill_name}' not found. Available skills: {available}"

# Create skill middleware
class SkillMiddleware(AgentMiddleware):
    """Middleware that injects skill descriptions into the system prompt."""

    # Register the load_skill tool as a class variable
    tools = [load_skill]

    def __init__(self):
        """Initialize and generate the skills prompt from SKILLS."""
        # Build skills prompt from the SKILLS list
        skills_list = []
        for skill in SKILLS:
            skills_list.append(
                f"- **{skill['name']}**: {skill['description']}"
            )
        self.skills_prompt = "\n".join(skills_list)

```

```

def wrap_model_call(
    self,
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """Sync: Inject skill descriptions into system prompt."""
    # Build the skills addendum
    skills_addendum = (
        f"\n\n## Available Skills\n\n{self.skills_prompt}\n\n"
        "Use the load_skill tool when you need detailed information "
        "about handling a specific type of request."
    )

    # Append to system message content blocks
    new_content = list(request.system_message.content_blocks) + [
        {"type": "text", "text": skills_addendum}
    ]
    new_system_message = SystemMessage(content=new_content)
    modified_request = request.override(system_message=new_system_message)
    return handler(modified_request)

# Initialize your chat model (replace with your model)
# Example: from langchain_anthropic import ChatAnthropic
# model = ChatAnthropic(model="claude-3-5-sonnet-20241022")
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4")

# Create the agent with skill support
agent = create_agent(
    model,
    system_prompt=(
        "You are a SQL query assistant that helps users "
        "write queries against business databases."
    ),
    middleware=[SkillMiddleware()],
    checkpoint=InMemorySaver(),
)

# Example usage
if __name__ == "__main__":
    # Configuration for this conversation thread
    thread_id = str(uuid.uuid4())
    config = {"configurable": {"thread_id": thread_id}}

    # Ask for a SQL query
    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": (
                        "Write a SQL query to find all customers "
                        "who made orders over $1000 in the last month"
                    )
                },
            ],
        }
    )

```

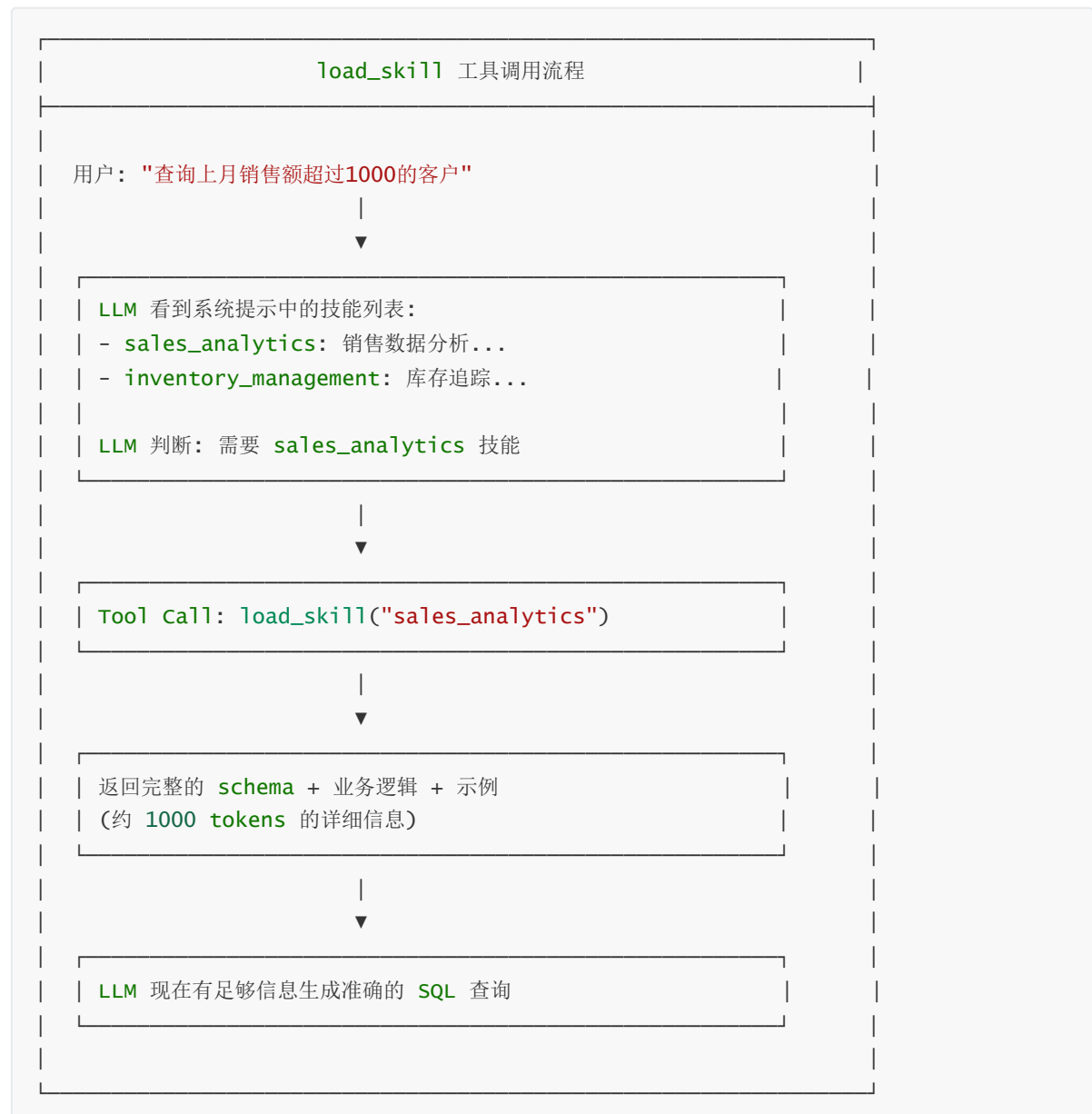
```

    ]
    },
    config
)

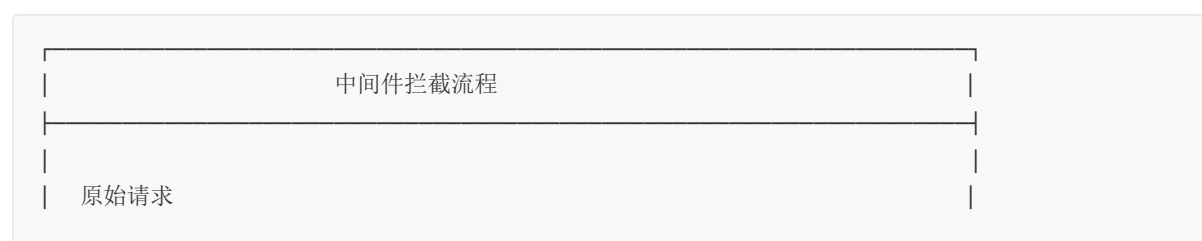
# Print the conversation
for message in result["messages"]:
    if hasattr(message, 'pretty_print'):
        message.pretty_print()
    else:
        print(f"{message.type}: {message.content}")

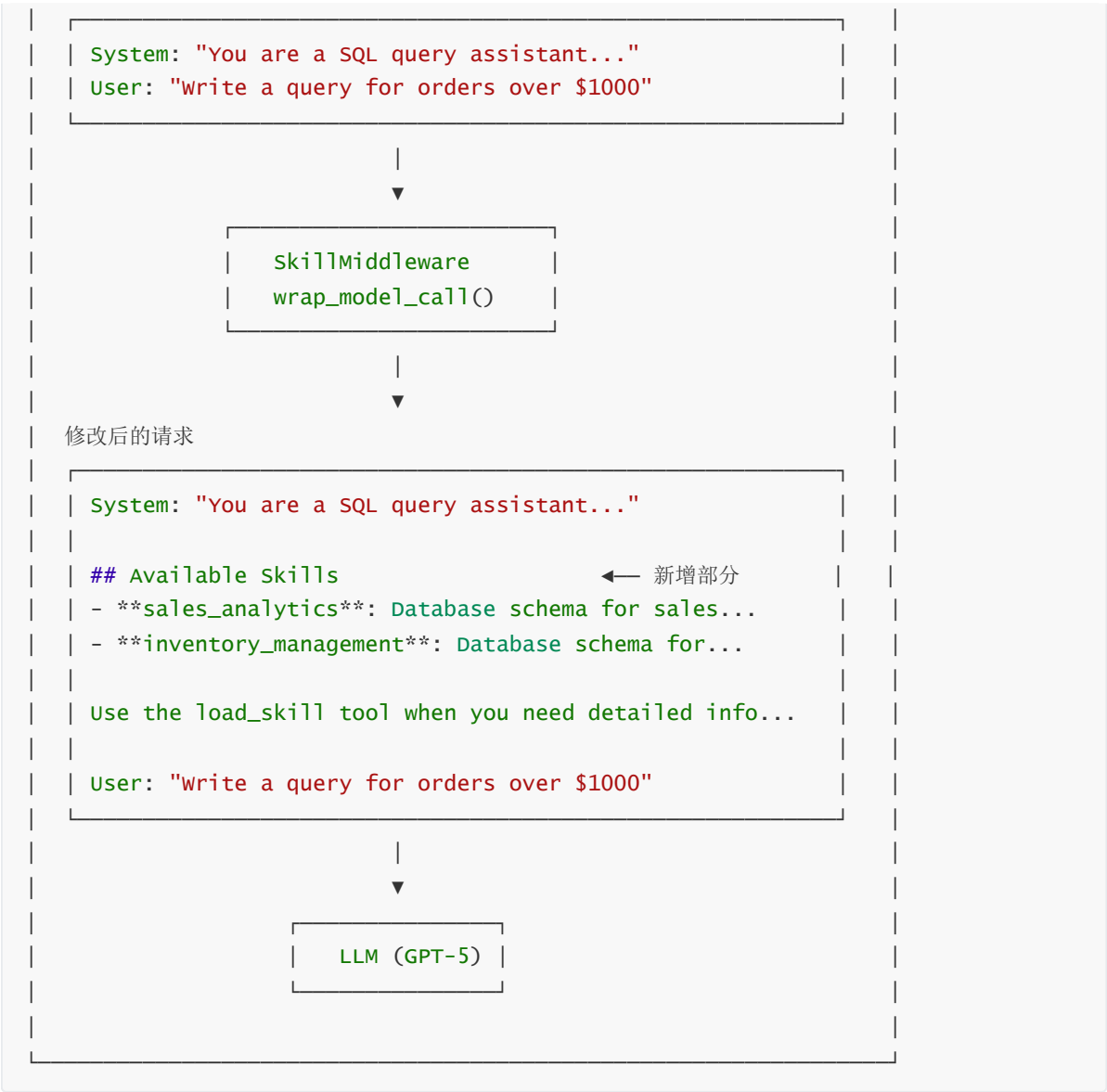
```

工具调用流程



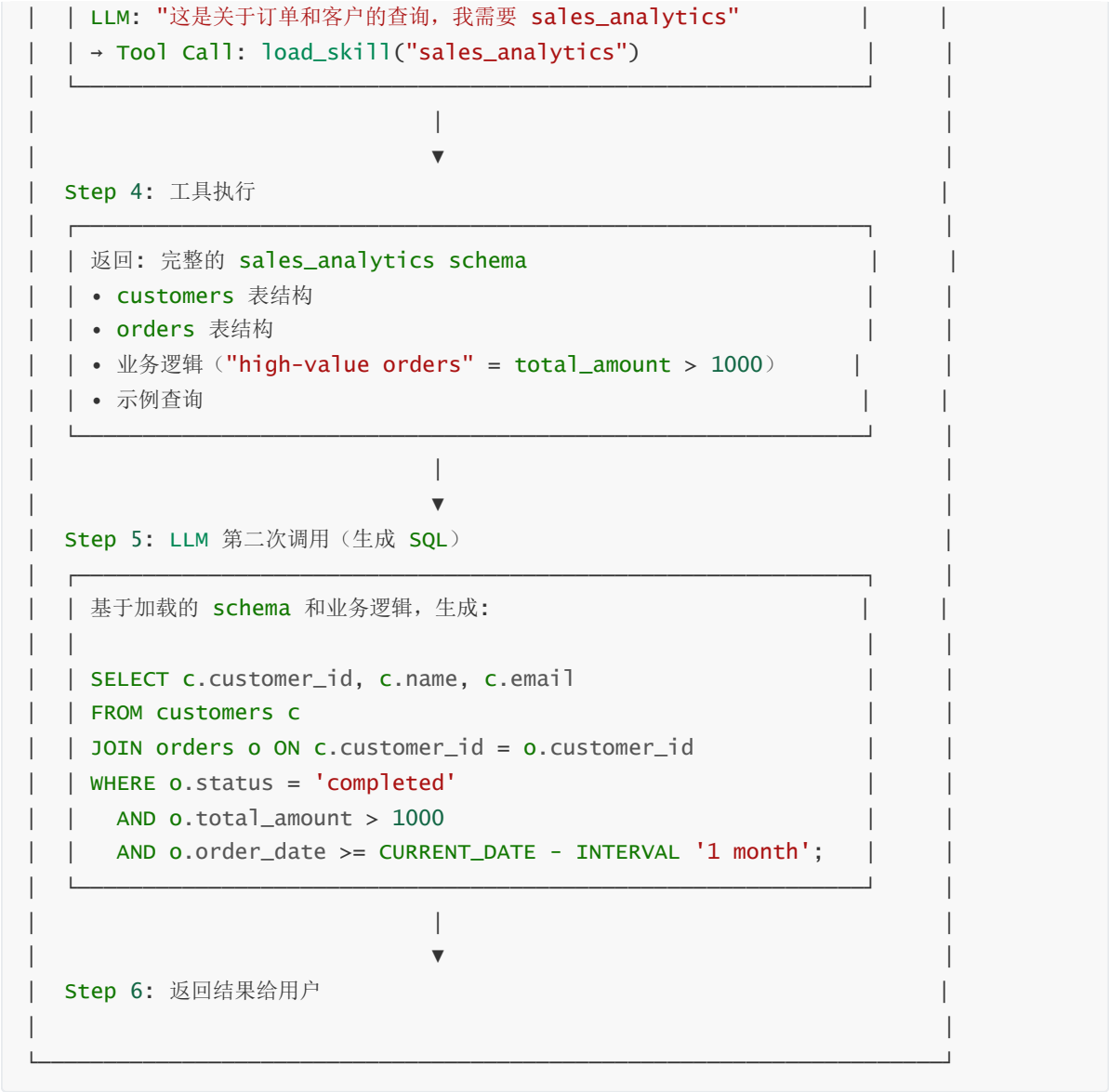
中间件工作原理





完整执行流程





核心设计模式总结



```
| |─ thread_id 标识不同会话
| |─ 支持多轮对话
|
```

七、行业动态与生态演进

7.1 标准化进程与厂商支持

Agent Skills 虽然由 Anthropic 提出，但其设计理念正在影响整个行业。

Anthropic Claude:

- Claude Desktop 和 Claude Code 原生支持 Skills
- 提供官方 SDK 和开发工具
- 维护官方技能库

OpenAI 的响应: 虽然 OpenAI 尚未官方采用 "Skills" 这个术语，但在 2025 年 3 月的更新中，ChatGPT 引入了类似的概念：

- **Custom Instructions 增强:** 支持更复杂的多步骤指令
- **Memory 与 Context Profiles:** 允许用户保存和复用特定领域的知识
- **GPTs 的"知识库"功能:** 可以附加文档和脚本，按需加载

这些功能本质上是 Skills 理念的不同实现形式。

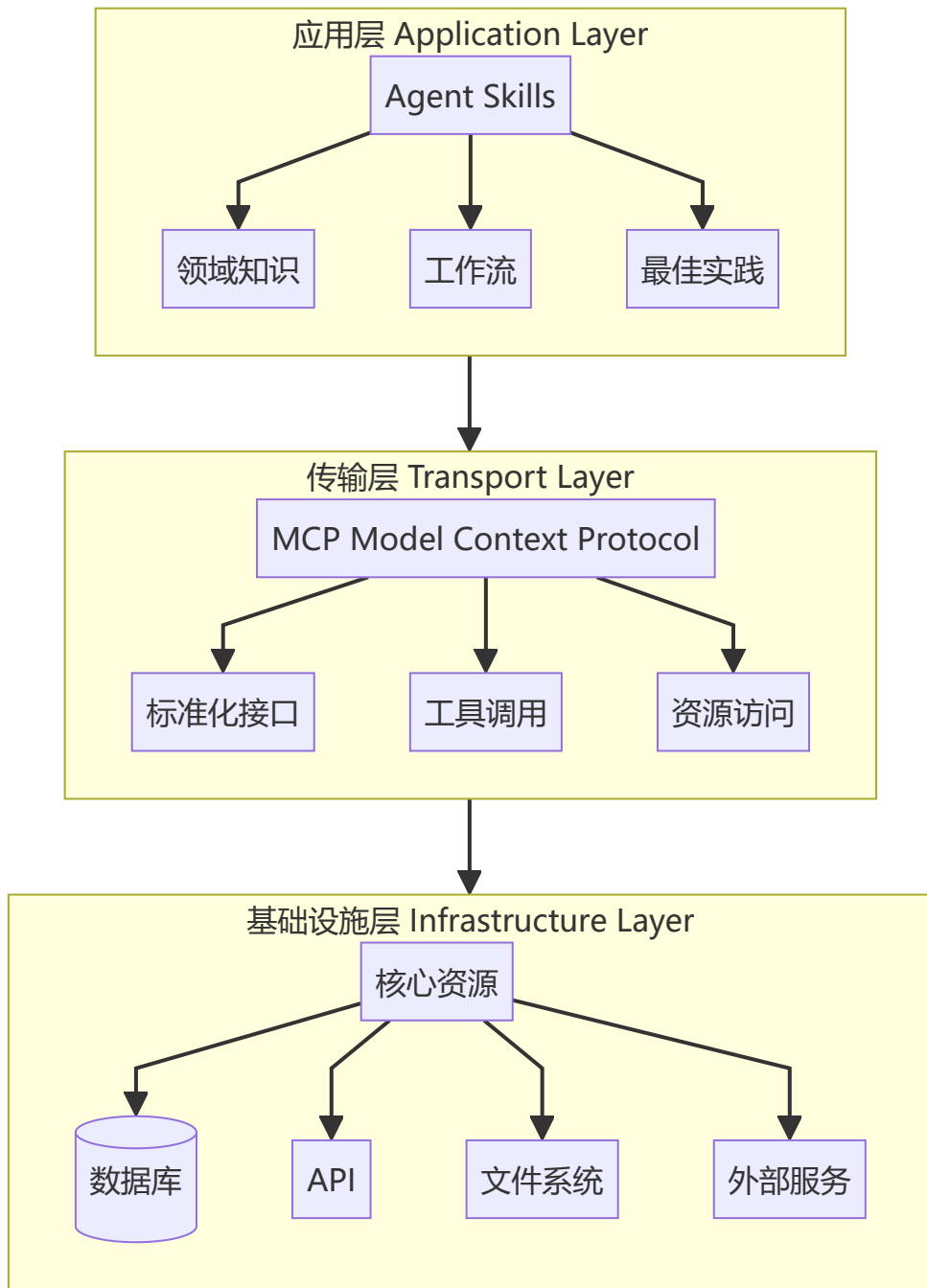
Google Vertex AI: Google 在 Gemini 模型中引入了 "Grounding with Functions"，允许开发者定义 "函数包" (Function Packages)，每个包包含：

- 函数定义 (类似 MCP 的 tools)
- 使用指南 (类似 Skills 的 instructions)
- 示例 (examples)

这种设计与 Skills + MCP 的混合架构高度相似

7.2 分层架构的必然性

综合各方观点，认为：**Skills 和 MCP 代表了智能体架构中两个必然分离的层级**。随着智能体系统的复杂度增加，这种分层是不可避免的：



这与传统软件架构的演进路径完全一致（从单体到分层到微服务），只是在 AI 领域重新演绎了一遍。

7.3 标准化的趋势

随着行业对智能体技术的重视，我们预见以下趋势：

1. 协议融合

未来可能出现统一的智能体能力描述协议，融合 MCP 的连接性和 Skills 的知识表达：

```
# 未来的统一协议示例（假想）
apiVersion:agent.io/v1
kind:Capability
metadata:
  name:enterprise-data-analysis
spec:
  transport:
    protocol:mcp
```

```
server:database-mcp-server
tools:[query,schema]
knowledge:
type:skill
workflow:data-analysis-workflow.md
examples:examples/
```

2. 市场化与生态系统

类似于 NPM、PyPI，未来可能出现智能体能力的包管理系统：

```
# 假想的未来命令
agent-cli install @anthropic/frontend-design-skill
agent-cli install @google/data-analysis-suite
agent-cli install @openai/code-review-assistant
```

开发者可以发布、分享、售卖自己的 Skills 和 MCP 服务器，形成繁荣的生态系统。

3. 自动化能力发现

智能体可能发展出自动发现和学习新能力的机制：

```
# 未来的智能体可能具备自主学习能力
agent = SelfEvolvingAgent() # 智能体在执行任务时发现缺少某种能力
response = agent.run("生成 3D 建模文件")
# 智能体自动搜索并安装相关 Skill
# [内部日志] 检测到未知任务类型：3D建模
# [内部日志] 搜索技能库...发现 "blender-3d-modeling" skill
# [内部日志] 请求用户授权安装...已授权
# [内部日志] 技能安装完成，重新执行任务
```

八、挑战与风险

与此同时，我们也需要警惕潜在的风险：

安全性挑战：

- Skills 包含可执行脚本，存在代码注入风险
- MCP 服务器可能暴露敏感数据接口
- 第三方技能的可信度难以验证

上下文污染：

- 随着 Skills 数量增加，即使是元数据也可能占用大量上下文
- 需要更智能的技能索引和检索机制

碎片化风险：

- 虽然 MCP 正在标准化，但 Skills 格式尚未统一
- 不同厂商可能推出不兼容的 Skills 规范

九、Agent Skills 和 MCP 总结与选型

Agent Skills 和 MCP 代表了智能体技术栈中两个关键的抽象层：

- **MCP (Model Context Protocol)**：解决"连接性"问题，是智能体与外部世界交互的标准化接口，相当于"神经系统"或"双手"
- **Agent Skills**：解决"能力"问题，是领域知识和工作流的封装，相当于"大脑皮层"或"操作手册"

两者不是竞争关系，而是互补关系：

	 MCP (Model Context Protocol)	 Agent Skills
核心价值	 标准化访问接口	 程序性知识封装
抽象层级	 Transport Layer 传输层/连接层	 Application Layer 应用层/逻辑层
解决问题	 How to Connect 如何连接外部资源	 How to Use 如何有效使用这些资源
技术形式	 JSON-RPC 2.0 协议 <pre>{ "jsonrpc": "2.0", "method": "...", }</pre>	 Markdown + Scripts 自然语言+脚本
上下文策略	 Eager Loading 急切加载（一次性全部）	 Lazy Loading 惰性加载（渐进式按需）
目标用户	 Developers 开发者	 Everyone 所有人（开发者+领域专家+业务人员）
 互补关系，共同构建完整的智能体能力体系		
Token效率对比: MCP: 16,000 tokens → Skills: 500 tokens (节省96%) ↑		

图 5 MCP 与 Agent Skills 全面对比总结

关键洞察：

1. **分层架构是必然趋势**：随着智能体系统复杂度增加，"连接层"和"知识层"的分离是不可避免的
2. **上下文效率是核心矛盾**：Skills 的渐进式披露机制将 token 消耗降低 90% 以上，这是其最大的技术优势
3. **领域知识的民主化**：Skills 让非开发者也能贡献智能体能力，这将极大拓展 AI 应用的边界
4. **混合架构是最佳实践**：在企业级应用中，MCP 提供基础设施连接，Skills 提供业务逻辑，两者结合才能构建高效、可维护的智能体系统

实践建议：

- 对于**外部服务连接**（数据库、API、云服务），优先使用 MCP
- 对于**复杂工作流**（多步骤任务、领域专业知识），优先使用 Skills
- 在**上下文受限**的场景（长对话、大量工具），使用 Skills 进行渐进式管理
- 构建**企业级智能体**时，采用 MCP + Skills 的分层架构

智能体技术仍在快速演进中。MCP 已成为连接层的事实标准，Skills 的理念也在影响整个行业。掌握这两种技术，将帮助你在 AI 浪潮中构建更强大、更实用的智能体应用。

十、Skills 社区库

<https://skillsllm.com/>

<https://skills.sh/>

<https://github.com/affaan-m/everything-claude-code>

参考资料

1. Anthropic Agent Skills 官方文档: <https://docs.anthropic.com/en/docs/agent-skills>
2. Anthropic Skills GitHub 仓库: <https://github.com/anthropics/skills>
3. Model Context Protocol 规范: <https://modelcontextprotocol.io/>
4. Anthropic 博客: Improving Frontend Design Through Skills: <https://www.claude.com/blog/improving-frontend-design-through-skills>